DATA STRUCTURES AND ALGORITHMS

UNIT 5 PROJECT REPORT

Represent University Courses and Their Prerequisite Relationships Using a Directed Graph and Apply DFS to Explore Reachable Courses

Project	University Course Prerequisite Graph Using DFS
Programming Language	C++
Student	Patkal Payal
Student id	226

A C++ programming project demonstrating directed graph representation and DFS traversal for exploring university course dependencies.

1. Introduction

Data structures are used to store and manage information in computer programs. Graphs are especially useful when representing relationships between objects. In a university, some courses depend on knowledge gained from other courses. These relationships can be represented using a directed graph, where each course is a vertex and each directed edge connects a prerequisite course to a dependent course.

This project uses C++ to create a graph of university courses. The user enters course names and prerequisite relationships, then applies Depth First Search (DFS) to explore courses reachable from a selected course. The project demonstrates directed graphs, adjacency lists, recursion and graph traversal.

2. Problem Statement

A university offers different courses, and some courses require students to complete other courses before studying them. Managing these relationships is important for understanding course dependencies. The project represents courses as vertices and prerequisite relationships as directed edges. It accepts course names and relationships from the user, creates the graph, and displays reachable courses using DFS traversal.

3. Objectives

- Understand directed graphs and how edges represent relationships.
- Represent prerequisite courses and dependent courses.
- Implement Depth First Search to explore reachable courses.
- Use an adjacency list to store graph connections.
- Display reachable courses while excluding the selected starting course.

4. Theory and Key Concepts

4.1 Directed Graph

A directed graph is a collection of vertices connected by edges with a specific direction. In this project, each vertex represents a university course and each directed edge indicates that one course is a prerequisite for another.

4.2 Adjacency List

An adjacency list stores the neighbouring vertices of each course. The C++ program creates the list using a vector of vectors: `vector<vector<int> > graph(n);`

4.3 Depth First Search (DFS)

DFS explores one path as deeply as possible before returning to explore other available paths. It marks each course as visited, displays its name, checks its neighbours and recursively visits each unvisited neighbour.

5. Working of the Program

The program first accepts the number of courses and their names. It then accepts prerequisite relationships and stores them in the adjacency list. The user enters a starting course, and the program checks whether that course exists. If it exists, DFS explores connected courses and displays them in traversal order. If the course is not found, an error message is displayed.

5.1 Role of the Visited Array

The visited array records whether a course has already been visited. All courses are initially unvisited. When DFS visits a course, its status changes to visited. This prevents repeated visits and avoids infinite traversal when cycles exist in the graph.

6. Algorithm

- Step 1: Start.
- Step 2: Enter the number of courses and their names.
- Step 3: Create an adjacency list to store the directed graph.
- Step 4: Enter the number of prerequisite relationships.
- Step 5: Read each prerequisite course and its dependent course.
- Step 6: Add a directed edge from the prerequisite course to the dependent course.
- Step 7: Enter the selected starting course.
- Step 8: Check whether the starting course exists in the graph.
- Step 9: If the course does not exist, display “Course not found” and stop.
- Step 10: Initialize all courses as unvisited.
- Step 11: Find the courses directly connected to the starting course.
- Step 12: Apply DFS to each unvisited neighbouring course.
- Step 13: Mark the current course as visited and display its name.
- Step 14: Explore the unvisited neighbouring courses recursively.
- Step 15: Continue until all reachable courses have been visited.
- Step 16: Display the traversal completion message.
- Step 17: Stop.

7. System Design

The program is organized into two main stages: graph construction and traversal. Graph construction maps course names to vertices and adds directed edges for each prerequisite relationship. Traversal begins from the selected course’s direct neighbours and calls DFS recursively for each unvisited course.

8. Complexity Analysis

Operation	Time Complexity	Reason
Build adjacency list	O(nm) in this implementation	Each relationship searches the course list to find both course indices.
DFS traversal	O(V + E)	Each reachable vertex and edge is explored at most a constant number of times.
Graph storage	O(V + E)	Stores all vertices and directed edges in the adjacency list.

V = number of vertices (courses); E = number of directed edges (prerequisite relationships).

9. Flowchart

The flowchart shows the sequence for entering course data, constructing the graph, validating the starting course, initializing the visited array and performing recursive DFS traversal.

5.Flowchart:

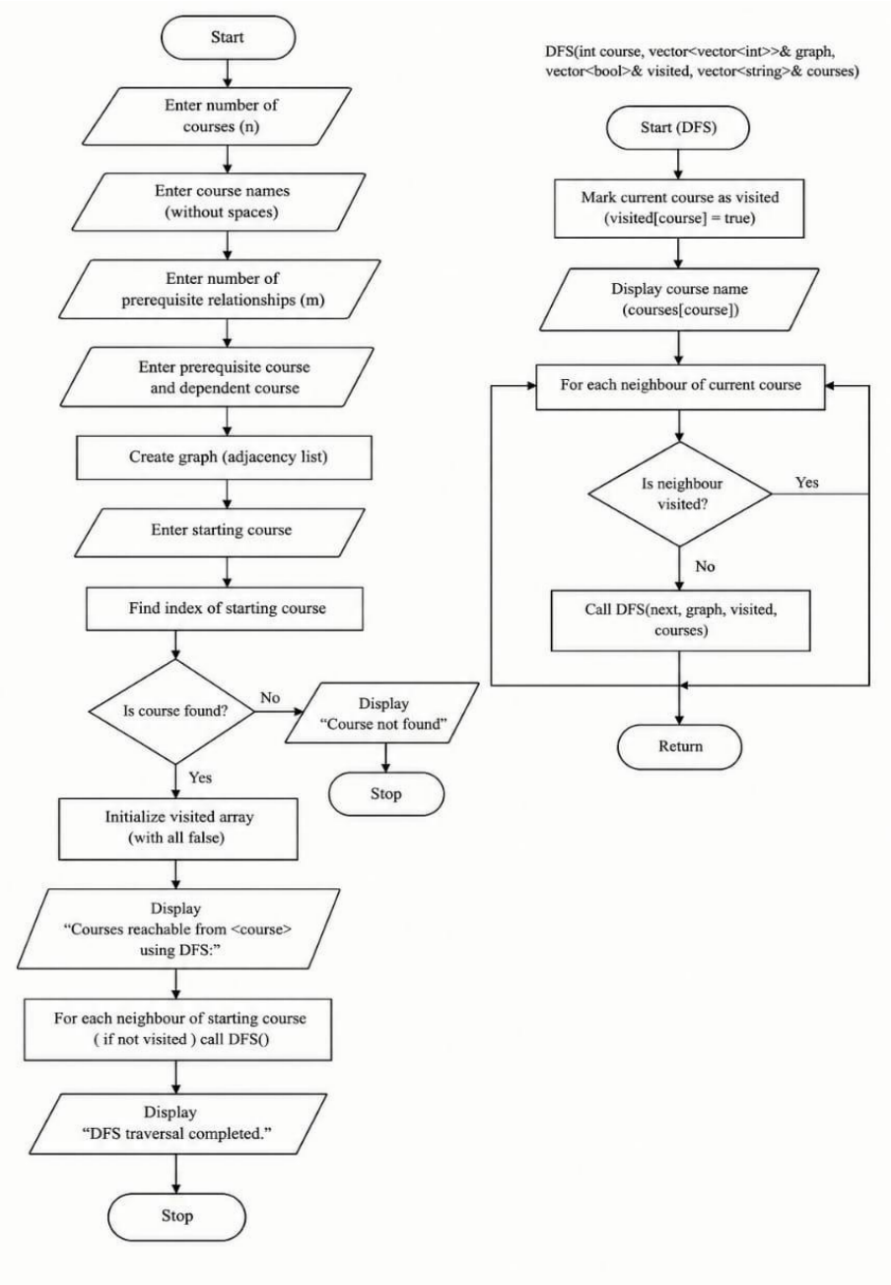


Figure 1. Program flow and recursive DFS process (retained from the source document).

10. C++ Source Code

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

void DFS(int course, vector<vector<int> > &graph,
         vector<bool> &visited, vector<string> &courses) {
    visited[course] = true;
    cout << courses[course] << endl;

    for (int i = 0; i < graph[course].size(); i++) {
        int next = graph[course][i];
        if (!visited[next]) {
            DFS(next, graph, visited, courses);
        }
    }
}

int main() {
    int n, m;

    cout << "Enter number of courses: ";
    cin >> n;

    vector<string> courses(n);

    cout << "Enter course names (without spaces):" << endl;
    for (int i = 0; i < n; i++) {
        cin >> courses[i];
    }

    vector<vector<int> > graph(n);

    cout << "Enter number of prerequisite relationships: ";
    cin >> m;

    cout << "Enter prerequisite course and dependent course:"
         << endl;

    for (int i = 0; i < m; i++) {
        string prerequisite, dependent;
        cin >> prerequisite >> dependent;

        int u = -1, v = -1;

        for (int j = 0; j < n; j++) {
            if (courses[j] == prerequisite)
```

10. C++ Source Code (continued)

```
                u = j;

            if (courses[j] == dependent)
                v = j;
        }

        if (u != -1 && v != -1) {
            graph[u].push_back(v);
        } else {
            cout << "Invalid course name. Relationship skipped."
                 << endl;
        }
    }

    string selected;
    cout << "Enter the starting course: ";
    cin >> selected;

    int start = -1;

    for (int i = 0; i < n; i++) {
        if (courses[i] == selected) {
            start = i;
            break;
        }
    }

    if (start == -1) {
        cout << "Course not found." << endl;
        return 0;
    }

    vector<bool> visited(n, false);

    cout << "\\nCourses reachable from " << selected
         << " using DFS:" << endl;

    for (int i = 0; i < graph[start].size(); i++) {
        int next = graph[start][i];

        if (!visited[next]) {
            DFS(next, graph, visited, courses);
        }
```

11. Sample Input

```
7
Programming
Mathematics
DataStructures
Algorithms
OperatingSystems
Database
AI
6
Programming DataStructures
Mathematics DataStructures
DataStructures Algorithms
DataStructures OperatingSystems
Programming Database
Algorithms AI
Programming
```

12. Sample Output

```
Enter number of courses: 7
Enter course names (without spaces):
Programming
Mathematics
DataStructures
Algorithms
OperatingSystems
Database
AI
Enter number of prerequisite relationships: 6
Enter prerequisite course and dependent course:
Programming DataStructures
Mathematics DataStructures
DataStructures Algorithms
DataStructures OperatingSystems
Programming Database
Algorithms AI
Enter the starting course: Programming

Courses reachable from Programming using DFS:
DataStructures
Algorithms
AI
OperatingSystems
Database

DFS traversal completed.
```

is directly connected to Programming; Algorithms is connected to DataStructures; AI is connected to Algorithms; OperatingSystems is also connected to DataStructures; and Database is directly connected to Programming. Mathematics is not displayed because there is no directed path from Programming to Mathematics.

14. Conclusion

The University Course Prerequisite Graph project demonstrates the use of directed graphs and Depth First Search in C++. Course names are represented as vertices, and prerequisite relationships are stored as directed edges using an adjacency list. The program accepts course details from the user and explores reachable courses from a selected course. The visited array helps avoid repeated visits. The project could be extended to detect circular dependencies and suggest possible learning paths.

15. References

- Course notes and classroom material on graphs, DFS and C++ data structures.
- C++ standard library documentation for vector and string.

Prepared as a formatted Unit 5 project report based on the supplied source document.